

Revision — 1.4.3 Boolean Algebra

OCR H446 — one-page revision summary

Must know

- **Gates:** NOT $\neg A$ (invert); AND $A \cdot B$ (both 1); OR $A+B$ (either 1); XOR $A \oplus B$ (inputs **differ**); NAND $\neg(A \cdot B)$; NOR $\neg(A+B)$. **NAND and NOR are universal** — any circuit can be built from either alone.
- **De Morgan's laws:** $\neg(A \cdot B) = \neg A + \neg B$ and $\neg(A+B) = \neg A \cdot \neg B$. "Break the bar, change the sign" — negate **each** variable **and** swap AND $\leftrightarrow$ OR.
- **Core identities:** Identity $A \cdot 1=A$, $A+0=A$; Null $A \cdot 0=0$, $A+1=1$; Idempotent $A \cdot A=A$, $A+A=A$; Complement $A \cdot \bar{A}=0$, $A+\bar{A}=1$; Double negation $\neg(\neg A)=A$.
- **Distributive:** $A \cdot (B+C) = A \cdot B + A \cdot C$ and $A+(B \cdot C) = (A+B) \cdot (A+C)$. **Absorption:** $A + A \cdot B = A$ and $A \cdot (A+B) = A$.
- **Simplify** expressions by applying identities step by step; **tidy** with double negation. Use to reduce gate count.
- **Karnaugh (K-)maps:** plot outputs, group adjacent **1s** in blocks that are **powers of two** (1, 2, 4, 8...), as **large** as possible, may wrap/overlap $\rightarrow$ minimal sum-of-products.
- **Half adder** (2 bits, no carry-in): **Sum** = $A \oplus B$, **Carry** = $A \cdot B$.
- **Full adder** (A, B, Cin) = two half adders + OR: **Sum** = $A \oplus B \oplus \text{Cin}$, **Cout** = $(A \cdot B) + (\text{Cin} \cdot (A \oplus B))$. Chain them $\rightarrow$ **ripple-carry adder** for multi-bit addition.
- **D-type flip-flop:** edge-triggered **1-bit memory** (sequential logic). On the active **clock edge** it stores input **D** and presents it at **Q** until the next edge; synchronises data to the clock. Uses: registers, counters, memory cells.
- **Combinational** logic = output depends only on current inputs; **sequential** logic = output depends on inputs **plus stored state**.

Key terms

Term	Definition
XOR	Gate outputting 1 only when its two inputs differ
Universal gate	Gate (NAND or NOR) from which any circuit can be built
De Morgan's law	$\neg(A \cdot B) = \neg A + \neg B$, $\neg(A+B) = \neg A \cdot \neg B$ — negate each term and swap operator
Absorption law	$A + A \cdot B = A$; $A \cdot (A+B) = A$
Karnaugh map	Grid grouping adjacent 1s (powers of 2) to minimise an expression
Half adder	Adds 2 bits $\rightarrow$ Sum ($A \oplus B$), Carry ($A \cdot B$); no carry-in
Full adder	Adds 3 bits (A, B, Cin) $\rightarrow$ Sum + Cout; chainable
D-type flip-flop	Edge-triggered 1-bit memory storing D on a clock edge

Worked example / how to

Simplify $\neg(\neg A + B)$ using De Morgan's law + double negation: - Apply De Morgan's to the outer bar (break bar, change + to $\cdot$, negate each term): $\neg(\neg A + B) = \neg(\neg A) \cdot \neg B$. - Apply double negation $\neg(\neg A)$

= $A : \rightarrow A \cdot \neg B$. - Result: $A \cdot \neg B$.

Exam tips

- De Morgan's: negate **each** variable **and** change the operator — not just one side; finish by tidying double negations.
- State **Sum** = $A \oplus B$ and **Carry** = $A \cdot B$ exactly; the half adder has **no carry-in**, the full adder adds **three** bits.
- For simplification, **name the law** used at each step (e.g. absorption, distributive) — it earns marks.
- K-map groups must be **powers of two** and **as large as possible**; they may wrap around and overlap.